Smart Study Planner - System Documentation

This document provides a detailed breakdown of every technology, library, file, database schema, and custom programming logic used to build the **Smart Study Planner** application.

---

1. Technical Stack Overview

The application is built using a modern, lightweight **Web Stack** designed for speed, responsiveness, and zero-friction hosting:

- **Frontend (User Interface)**:
  - **HTML5**: Semantic markup structuring the app pages and Single Page Application (SPA) views.
  - **Vanilla CSS3**: Tailored stylesheet featuring custom CSS variables, a modern glassmorphic design system, responsiveness, fluid grids, and zero-gravity animations.
  - **Vanilla JavaScript (ES6+)**: Powers client-side state, event listeners, dynamic UI rendering, local storage configurations, and graphics processing.
  - **Chart.js (v4.x)**: Handles visual plotting of weekly focus curves and targets on the dashboard.
  - **Lucide Icons**: Renders modern SVG icons (trash cans, checkmarks, open-book indicators) on the UI.
- **Backend (Server)**:
  - **Node.js**: The JavaScript runtime environment powering the server.
  - **Express.js**: Rest API framework serving static frontend assets and routing authenticated requests.
  - **JSON Web Tokens (JWT)**: Signs and authenticates session tokens to lock down protected API routes.
  - **Bcrypt.js**: Cryptographically hashes and salts user passwords for secure storage.
- **Database System (Hybrid)**:
  - **SQLite (`sqlite3`)**: Zero-config, file-based database for local offline testing (`database.sqlite`).
  - **PostgreSQL (`pg`)**: Enterprise-grade cloud relational database used on Render (via **Neon.tech** or **Supabase**) to ensure permanent data persistence.
- **AI Integration**:
  - **OpenAI Node SDK**: Connects the chat widget and Feynman Room to OpenAI's GPT models, with dynamic, built-in simulated fallbacks.

---

2. Directory & File Architecture

Here is the exact purpose of every file in the project folder:

```
■■■ database.sqlite          # Local file containing the SQLite database (auto-generated)
■■■ dashboard-data.js        # Holds mock metrics (focus times, study trends) for the dashboard charts
■■■ dashboard.html           # Main dashboard user interface (authenticated view)
■■■ dashboard.js             # Controller for dashboard.html: Pomodoro timer, Chart.js plotter, stats loade
■■■ feynman.html             # The Feynman active recall explanation workspace page
■■■ feynman.js               # Logic for capturing descriptions, evaluating text, and grading topics
■■■ index-dashboard.js       # SPA router for index.html, mascot chatbot logic, local-mode banner toggles
■■■ index.html               # Public landing page and guest dashboard SPA
■■■ login.html               # Login page gate
■■■ signup.html              # Account registration gate
■■■ package.json             # NPM package dependencies configuration
■■■ package-lock.json        # Strict versioning tree for npm dependencies
■■■ render.yaml              # Blueprint file for Render deployment environment auto-detection
■■■ robot_mascot.png         # High-resolution mascot artwork for the floating chatbot (Nova)
■■■ server.js                # Express API server, routes, and Hybrid Database client router
■■■ spaced-repetition.js     # Scheduling engine (1-3-7-30 day spaced interval matrix)
■■■ style.css                # Global stylesheet containing core design systems, themes, and animations
■■■ theme.js                 # Local theme manager controlling dark/light mode toggle states
```

---

3. Database Schema

The database supports a relational structure mapped dynamically to either SQLite (locally) or PostgreSQL (in production):

1. `users` Table

Stores user accounts.

- **`id`**: Primary Key (Auto-incrementing Integer).
- **`username`**: Unique String (Text).
- **`password`**: Hashed String (Text).

2. `tasks` Table

Stores tasks created in the study timeline.

- **`id`**: Primary Key (Auto-incrementing Integer).
- **`userId`**: Foreign Key (Integer) linking to the user who created the task.
- **`text`**: The task description (Text).
- **`completed`**: Boolean flag (stored as 0/1 in SQLite, true/false in PostgreSQL).
- **`tag`**: Subject categorization label (Text).

3. `stats` Table

Tracks focus statistics.

- **`userId`**: Primary Key (Integer) matching the user's ID.
- **`totalStudyTime`**: Total study time accumulated in seconds (Integer).

- `sessionsCompleted`: Number of completed Pomodoro study blocks (Integer).

4. `calendarStore` (Stored in browser `localStorage`)

Tracks spaced repetition milestone events:

- `id`: Unique event string key.
- `topicId`: References the associated study topic.
- `title`: Text (e.g., `"Review 1: Subject - Topic"`).
- `startDateTime` / `endDateTime`: ISO timestamp strings.
- `status`: `'pending'` or `'completed'`.
- `intervalStep`: Step number (`1`, `2`, `3`, or `4` mapping to 1, 3, 7, 30 days).

---

4. Custom Programming Logics

Several key features utilize customized programming methods to eliminate user friction and maintain smooth operations:

A. The Hybrid Database Client Router (`server.js`)

Instead of duplicating the codebase or forcing you to install complex database engines on your computer, the backend uses a custom class wrapper `HybridDatabase` that automatically detects its environment:

- If the environment variable `DATABASE_URL` is found (e.g., on Render), it instantiates a PostgreSQL connection pool (`pg.Pool`) and converts all query bindings from SQLite format (`?`) to PostgreSQL format (`$1, $2`).
- If `DATABASE_URL` is missing (locally), it falls back to the local `database.sqlite` file.

B. Client-Side Mascot Background Removal (`index-dashboard.js` & `dashboard.js`)

To avoid loading heavy image editing libraries on the server, the mascot background is made transparent dynamically in the user's browser using HTML5 Canvas:

1. The original mascot image (`robot_mascot.png`) is drawn onto an offscreen `<canvas>`.
2. A **Breadth-First Search (BFS) Flood Fill** algorithm starts crawling from the outer edge coordinates.
3. Any pixels matching the background blue color (`#4B8BD2` / RGB `75, 139, 210`) within an RGB color tolerance threshold are converted to transparent pixels (alpha = 0).
4. This isolates the background while keeping any matching blue shades inside Nova's hair/eyes intact because they are bounded by non-background pixels.
5. The canvas generates a transparent base64 PNG data URL to display.

C. Spaced Repetition Logic (`spaced-repetition.js`)

Implements the 1-3-7-30 day retention schedule:

- When a topic is marked mastered, the system calculates 4 review intervals starting from the current date (+1, +3, +7, and +30 days).

- To prevent midnight rollover bugs (e.g., days shifting due to timezones), it standardizes the scheduled start times to **9:00 AM local time**.

---

5. Core NPM Packages Used

- `express` (v5.2.1): Serves frontend files and REST API routes.
- `sqlite3` (v6.0.1): Simple embedded database driver for offline local runs.
- `pg` (v8.11.3): PostgreSQL client database driver for cloud database connection.
- `bcryptjs` (v3.0.3): Cryptographic password hashing.
- `jsonwebtoken` (v9.0.3): Signs and verifies user session JWT tokens.
- `cors` (v2.8.6): Enables cross-origin request configurations.
- `dotenv` (v17.4.1): Loads environment settings from a local `.env` file.
- `openai` (v6.34.0): Node wrapper for OpenAI ChatGPT integration.